

Safe Communication-Free MAPF

Chiawen Liao
chiawenl@andrew.cmu.edu

Thomas Chan
yuhsinch@andrew.cmu.edu

Abstract—Multi-agent path finding (MAPF) is a fundamental problem in multi-robot coordination. Centralized optimal solvers degrade in dense, large-scale settings, while decentralized learning-based approaches typically rely on inter-agent communication to resolve conflicts—an assumption that breaks down in subterranean environments, adversarial settings, or areas with heavy RF interference. We propose a communication-free decentralized MAPF system and address the problem through two complementary methods. First, we fine-tune MAPF-GPT on states where a centralized CS-PIBT shield intervenes, converting reactive shielding into proactive policy improvement; the fine-tuned policy reduces collisions by roughly 50% and cuts shield intervention rates by about 20% on average. Second, we introduce an uncertainty-aware planner that leverages a shared policy assumption and fixed priority orderings to simulate higher-priority agents, construct a cumulative occupancy risk map, and select actions that balance policy preference against collision risk. On the POGEMA benchmark, the vanilla MAPF-GPT-2M baseline achieves only 32% case success rate (CSR) at 64 agents; the planner alone raises this to 56%, and combining the fine-tuned policy with the planner reaches 68% CSR while reducing collisions by 56%.

I. INTRODUCTION & MOTIVATION

Multi-agent path finding (MAPF) is a fundamental problem in multi-robot coordination: given a set of agents in a shared environment, the goal is to find collision-free paths from their start locations to their assigned goals. This problem arises in automated warehouses, autonomous vehicle coordination, and drone fleet management, where dozens to thousands of robots must navigate efficiently without colliding.

Finding an optimal MAPF solution is NP-hard. Optimal solvers such as CBS [11] perform well in sparse environments but degrade in dense, large-scale settings. Sub-optimal methods—including EECBS [7], LaCAM [8], PIBT [9], and Windowed-MAPF [15]—offer bounded-suboptimal or complete solutions within tractable time budgets, but remain centralized and slow as agent count scales.

Decentralized learning-based approaches such as MAPF-GPT [1] address scalability by having each robot independently plan from local observations. However, this introduces a coordination challenge: without knowledge of others' intentions, resolving conflicts is difficult. Consider the configuration shown in Figure 1: two symmetric, equally optimal solutions exist, and two independently planning robots have no basis to prefer one over the other, making collisions likely. In safety-critical settings—such as a warehouse where one robot carries flammable materials—such collisions could have catastrophic consequences.

Works such as [12] and [13] mitigate this by re-centralizing

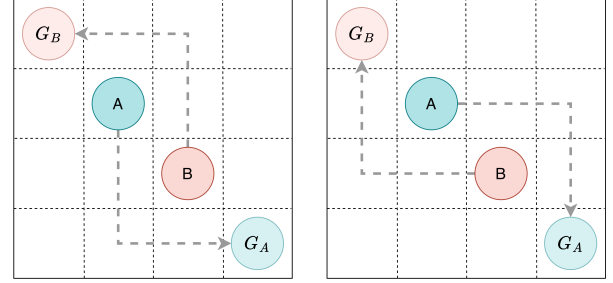


Fig. 1. A symmetric configuration where two independently planning agents have no basis to prefer one resolution over the other, making collisions likely.

coordination via inter-agent communication to run a conflict-resolution shield. This assumption is architecturally load-bearing: it breaks down in subterranean environments, adversarial settings with jamming, or dense warehouses with RF interference—precisely the safety-critical scenarios motivating MAPF.

In this work, we address a realistic variant of MAPF in which robots cannot communicate. We tackle this along two complementary directions: (1) learning a more conflict-aware policy via unsafe-state fine-tuning, and (2) designing a planning algorithm that reasons explicitly under uncertainty about unknown agent intentions.

II. RELATED WORK

A. MAPF

Multi-agent path finding (MAPF) methods can be broadly grouped into search-based, learning-based, and hybrid approaches. Search-based methods remain the standard choice when correctness and explicit conflict resolution are important, while learning-based methods improve scalability and inference speed. Hybrid methods aim to combine both, using learning for efficiency and planning or shielding for reliability.

1) *Search-based methods*: Search-based MAPF methods explicitly resolve inter-agent conflicts and are the dominant line of work when formal guarantees are required. Conflict-Based Search (CBS) [11] is a representative framework that separates global conflict handling from low-level single-agent planning. More recent methods such as LaCAM [8] improve scalability on large MAPF instances, while Windowed MAPF with Completeness Guarantees [15] shows that even receding-horizon planning can retain completeness.

2) *Learning-based methods*: Learning-based MAPF methods replace expensive online search with reactive policies.

Earlier works such as PRIMAL2 [5] and SCRIMP [16] show that decentralized policies can scale well and handle dense environments more efficiently at inference time. More recently, MAPF-GPT [1] and MAPF-GPT-DDG [2] demonstrate that large-scale imitation learning can further improve generalization and solution quality.

3) *Hybrid methods*: Hybrid methods combine learned policies with search-based correction or shielding mechanisms. Recent work shows that augmenting a learned local policy with CS-PIBT [13] can effectively resolve local collisions, and even simple imitation learning can become competitive once combined with such a planner [12].

B. Multi-agent interaction under uncertainty

Reasoning about other agents' behavior under uncertainty is a key challenge in decentralized settings. Sequential path planning (SPP) [3, 4] assigns strict priorities to agents, reducing joint planning complexity from exponential to linear, but assumes some state information is available. More recent approaches handle richer uncertainty: SN-CBFs [17] decompose multi-obstacle interactions into per-obstacle temporal sequences; contingency games [10] maintain a distribution over agent intents and plan contingent trajectories; and [6] uses adversarial RL to synthesize policies that account for the robot's evolving uncertainty at runtime. Our work draws on priority-based simulation in the spirit of SPP, but operates in a fully decentralized, communication-denied setting using only local observations and a shared policy.

III. PROBLEM FORMULATION

We formalize the problem of communication-free decentralized multi-agent path finding as follows.

a) *Environment*.: We are given an undirected graph $G = (V, E)$ representing the environment, where V is the set of traversable vertices and E is the set of edges between adjacent vertices. In grid-based settings, V corresponds to free cells and E connects cells reachable in one step.

b) *Agents*.: A set of N agents $\mathcal{A} = \{a_1, a_2, \dots, a_N\}$ operate in G . Each agent a_i has a start position $s_i \in V$ and a goal position $g_i \in V$. All start positions are distinct, and all goal positions are distinct.

c) *Actions and transitions*.: Time is discretized into timesteps. At each timestep t , each agent a_i selects an action from the action set $\{\text{WAIT}, \text{UP}, \text{DOWN}, \text{LEFT}, \text{RIGHT}\}$. The action transitions the agent to an adjacent vertex or keeps it at its current vertex.

d) *Observation model*.: Each agent a_i has a local field-of-view of radius r . At timestep t , agent a_i observes:

- The obstacle structure of G within radius r of its current position,
- The positions of all other agents within radius r ,
- Its own relative goal direction.

We denote the observation of agent a_i at timestep t as o_i^t .

e) *Assumptions*.: We make the following assumptions:

- 1) **Shared policy**. All agents execute the same deterministic policy $\pi : \mathcal{O} \rightarrow \text{Actions}$, where $\mathcal{O}$ is the observation space.
- 2) **Known priority ordering**. A fixed priority ordering $\sigma : \mathcal{A} \rightarrow \{1, 2, \dots, N\}$ is known to all agents prior to execution.
- 3) **Known map**. All agents have access to the full graph G .
- 4) **No communication**. Agents exchange no messages during execution. Each agent selects its action using only its local observation o_i^t , the shared policy π , the priority ordering σ , and the map G .

f) *Safety and completeness constraints*.: A valid execution must satisfy:

- **Vertex collision-free**: $\text{pos}(a_i, t) \neq \text{pos}(a_j, t)$ for all t and all $i \neq j$.
- **Edge collision-free**: For all t and all $i \neq j$, it is not the case that $\text{pos}(a_i, t) = \text{pos}(a_j, t+1)$ and $\text{pos}(a_j, t) = \text{pos}(a_i, t+1)$.
- **Completeness**: For each agent a_i , there exists a finite timestep T_i such that $\text{pos}(a_i, T_i) = g_i$.

g) *Objective*.: Minimize the sum-of-costs $\sum_{i=1}^N T_i$, where T_i is the first timestep at which agent a_i reaches its goal g_i .

h) *Distinction from prior work*.: In standard centralized MAPF, a single planner computes all paths with full knowledge of all agents. In existing decentralized learning-based MAPF (e.g., MAPF-GPT), each agent independently computes an action, but a communication-dependent collision shield (e.g., PIBT) resolves conflicts before execution. In our formulation, the safety and completeness constraints must be satisfied **by the policy and algorithm alone**, without any inter-agent message passing at runtime.

IV. PROPOSED APPROACH

Our approach proceeds in two stages. In Stage 1, we improve the base MAPF-GPT policy by fine-tuning on unsafe states mined via a PIBT safety shield (Section IV-A). This produces a conflict-aware policy π^* that is more robust in dense settings. In Stage 2, we use π^* as the shared policy within an uncertainty-aware planning framework (Section IV-B). Because all agents execute the same policy, agent i can use π^* to simulate the future behavior of higher-priority neighbors and construct a local risk map. The fine-tuned policy thus serves a dual role: it directly improves each agent's action quality, and it provides a more accurate model for predicting other agents' behavior during risk-aware planning.

A. Learning a self-safe policy via unsafe state mining

The first component of our approach aims to convert reactive safety correction into proactive policy learning. We start from the pretrained MAPF-GPT 2M policy, which can generate reasonable goal-directed actions but often produces local conflicts in dense multi-agent settings. To expose these failure modes,

we run MAPF-GPT together with a local CS-PIBT safety shield on dense maze, random, and warehouse maps. At each timestep, MAPF-GPT first proposes an action for every agent. The shield then checks the joint action for vertex conflicts, where two agents attempt to move into the same cell, and edge conflicts, where two agents attempt to swap positions. If a conflict is detected, CS-PIBT reassigns actions according to a local priority ordering and selects collision-free alternatives whenever possible.

A direct way to use this shield is to collect only the shield-modified states as unsafe states and train the policy to imitate the shield’s corrective action. However, our experiments show that this hard-state-only strategy can cause catastrophic forgetting: the model learns local collision-avoidance corrections but loses part of its original goal-reaching behavior. We therefore adopt an in-situ DAgger-style data collection strategy. Instead of recording only the states where the shield intervenes, we record all agent-level observation-action pairs from shielded dense rollouts. For each agent i at time t , let o_i^t be the local observation, $\hat{a}_i^t$ be the action proposed by MAPF-GPT, and $a_i^{t,\text{safe}}$ be the final action after the CS-PIBT shield. We add the pair $(o_i^t, a_i^{t,\text{safe}})$ to the training dataset:

$$\mathcal{D}_{\text{safe}} = \{(o_i^t, a_i^{t,\text{safe}})\}_{i,t}.$$

If the shield does not modify the action, then $a_i^{t,\text{safe}} = \hat{a}_i^t$, so the sample preserves the original navigation behavior of MAPF-GPT. If the shield changes the action, then $a_i^{t,\text{safe}}$ provides expert supervision for avoiding an imminent collision. This design is important because most states in a rollout are still normal navigation states, while only a small fraction correspond to actual shield corrections. In our final dataset, about 98% of the samples preserve the model’s original action and about 2% provide shield-corrected actions. This balance allows the model to learn collision-avoidance behavior without drifting away from the original path-following distribution.

We then fine-tune MAPF-GPT using supervised learning on $\mathcal{D}_{\text{safe}}$. The objective is to minimize the cross-entropy between the policy’s predicted action distribution and the shielded final action:

$$\theta^* = \arg \min_{\theta} \sum_{(o,a) \in \mathcal{D}_{\text{safe}}} \text{CE}(\pi_{\theta}(\cdot | o), a).$$

The resulting policy $\pi^* = \pi_{\theta^*}$ is trained not only to imitate shield corrections, but also to reproduce good dense-scenario navigation behavior from the shielded expert rollouts. In this sense, the shield acts as a temporary teacher during data collection, while the fine-tuned policy internalizes part of the shield’s safety behavior.

B. Uncertainty-Aware Planning via Risk Map

Although the learned policy π^* encourages conflict-free behaviour, it cannot guarantee the absence of collisions. We leverage the fixed priority ordering and local observability within radius r (Section III) to simulate higher-priority agents and build a cumulative occupancy risk map for action selection.

1) *Risk Map Construction:* For agent i , let $P = \{p_1, \dots, p_k\}$ be the higher-priority agents within radius r , ordered by priority. Since all agents share π^* , agent i simulates each agent in P for s timesteps. Actions with probability below ϵ are pruned and the distribution renormalised:

$$\tilde{\pi}(a | o) = \begin{cases} \frac{\pi^*(a|o)}{\sum_{a': \pi^*(a'|o) \geq \epsilon} \pi^*(a'|o)} & \text{if } \pi^*(a | o) \geq \epsilon, \\ 0 & \text{otherwise.} \end{cases} \quad (1)$$

For each simulated agent p_{ℓ} , we propagate marginal occupancy probabilities forward from its known position, initialising $r_{i_0 j_0 0}^{(p_{\ell})} = 1$:

$$r_{ijt}^{(p_{\ell})} = \sum_{(i',j') \in \mathcal{N}(i,j)} r_{i'j',t-1}^{(p_{\ell})} \cdot \tilde{\pi}(a_{(i',j') \rightarrow (i,j)} | o_{p_{\ell}}^{t-1}) \quad (2)$$

Simulation proceeds in priority order; each agent avoids cells with high cumulative risk from higher-priority agents. The cumulative risk map aggregates per-agent occupancies via element-wise maximum:

$$R_{ijt} = \max_{p_{\ell} \in P} r_{ijt}^{(p_{\ell})} \quad (3)$$

2) *Implicit Risk Propagation:* The raw risk map does not account for agents *passing through* cells en route to their goals. Using a distance-to-goal map d_{ij} (computed via backward Dijkstra), we propagate risk only toward the goal:

$$R_{xyt} \leftarrow \max(R_{xyt}, R_{ijt} \cdot \alpha^{d_{ij}-d_{xy}}) \quad \text{if } d_{xy} < d_{ij} \quad (4)$$

where $\alpha \in (0, 1)$ is a decay factor.

3) *Visited-State Penalty:* To prevent livelock—agents oscillating between cells blocked by higher-priority neighbours—we add a penalty inspired by Anytime PIBT [14]. Let $V_i(x, y) = |\{t \in H_i : \text{pos}(a_i, t) = (x, y)\}|$ count how often agent i visited cell (x, y) in the last W timesteps.

4) *Risk-Aware Action Selection:* For each candidate action a leading to cell (x, y) , the cost combines policy preference, collision risk, and revisitation penalty:

$$c(a) = -\lambda_1 \log \pi^*(a | o_i) + \lambda_2 \cdot R_{xy, \tau+1} + \lambda_3 \cdot V_i(x, y) \quad (5)$$

The agent samples from the softmax distribution $a^* \sim w(a) / \sum_{a'} w(a')$ where $w(a) = e^{-c(a)}$, preserving stochasticity to reduce the likelihood that multiple agents commit to the same conflicting action.

V. RESULTS

A. Baseline: MAPF-GPT 2M

We first evaluate the pretrained MAPF-GPT 2M policy as a purely learning-based decentralized MAPF baseline, without communication, explicit risk estimation, or runtime shielding. We report individual success rate (ISR), collective success rate (CSR), and average collisions per episode. ISR measures the fraction of agents that reach their goals, while CSR measures the fraction of episodes in which all agents succeed.

As results shown in Table I The baseline performs well in sparse or moderately dense random maps, achieving perfect CSR at 32 and 48 agents. However, its performance degrades

TABLE I
BASELINE MAPF-GPT 2M PERFORMANCE WITHOUT SHIELDING.

Map Type	Agents	ISR	CSR	Collisions/Ep.
random	32	1.000	1.000	33.8
random	48	1.000	1.000	112.8
random	64	0.894	0.700	363.2
mazes	32	0.950	0.500	67.0
mazes	48	0.706	0.100	347.6
mazes	64	0.508	0.000	701.3
warehouse	64	0.983	0.800	79.9
warehouse	96	0.945	0.200	303.9
warehouse	128	0.749	0.100	992.6

significantly as the environment becomes denser or more topologically constrained. For example, on maze maps, CSR drops from 0.500 at 32 agents to 0.100 at 48 agents and 0.000 at 64 agents. A similar trend appears in the warehouse setting, where CSR falls from 0.800 at 64 agents to 0.100 at 128 agents. At the same time, the number of collisions increases rapidly with density, reaching 701.3 collisions per episode on 64-agent mazes and 992.6 collisions per episode on 128-agent warehouse maps.

These results show that the pretrained policy can generate reasonable local navigation actions, but it lacks an explicit mechanism for resolving dense multi-agent conflicts. In particular, narrow maze corridors and high-density warehouse layouts create repeated vertex and edge conflicts that the neural policy alone cannot reliably avoid. This motivates our two complementary directions: first, fine-tuning the policy to internalize collision-avoidance behavior, and second, adding an uncertainty-aware planner that explicitly reasons about future occupancy risk.

B. Fine-tuned Policy (π^*)

We next evaluate the fine-tuned policy π^* obtained through in-situ DAgger fine-tuning. Instead of training only on shield-modified unsafe states, which can cause catastrophic forgetting, the final training pipeline collects all observation-action pairs from shielded dense episodes. Unmodified actions preserve the original navigation behavior, while shield-modified actions provide supervision for collision avoidance. The resulting dataset contains 941,568 samples from 180 dense episodes across random, maze, and warehouse maps.

Fine-tuning substantially improves collision avoidance while preserving, and in many cases improving, goal-reaching performance. Across the evaluated scenarios, π^* reduces collisions by roughly 50% on average. The improvement is most visible in dense and structured environments: on 48-agent mazes, collisions drop from 347.6 to 126.0 per episode, while ISR improves from 0.706 to 0.906 and CSR improves from 0.100 to 0.400. On 128-agent warehouse maps, collisions decrease from 992.6 to 398.6, while ISR improves from 0.749 to 0.959 and CSR improves from 0.100 to 0.400. These results suggest that π^* has learned part of the shield’s collision-avoidance behavior without losing the original policy’s navigation capability.

We also evaluate the fine-tuned policy together with the CS-PIBT shield to measure whether π^* reduces the need for runtime intervention. This setting compares the original MAPF-GPT 2M with shield against π^* with the same shield.

TABLE II
SHIELD INTERVENTION RATE BEFORE AND AFTER FINE-TUNING.

Map Type	Agents	Original + Shield	π^* + Shield	Change
random	32	32.1%	16.3%	-49%
random	48	43.0%	33.3%	-23%
random	64	66.7%	55.9%	-16%
mazes	32	22.5%	16.4%	-27%
mazes	48	47.8%	40.9%	-14%
mazes	64	60.6%	62.1%	+2%
warehouse	64	27.5%	21.2%	-23%
warehouse	96	43.3%	37.5%	-13%
warehouse	128	56.7%	52.5%	-7%

Like results shown in Table II and Figure 2, the fine-tuned policy reduces shield intervention in almost all settings, showing that it proactively avoids many conflicts that previously required external correction. The average shield trigger reduction is about 20% when excluding the nearly deadlocked 64-agent maze case. In warehouse scenarios, π^* with shielding maintains or improves reliability while requiring fewer shield activations: for example, at 128 agents, CSR improves from 0.900 to 1.000 while the shield rate decreases from 56.7% to 52.5%. The main exception is the 64-agent maze scenario, where narrow corridors create persistent topological deadlocks and the shield intervention rate slightly increases. This suggests that fine-tuning is effective for reducing local collision conflicts, but dense maze-like environments may still require more global coordination or explicit deadlock resolution.

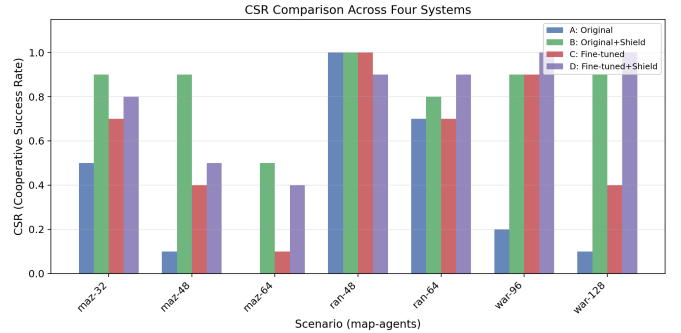


Fig. 2. CSR comparison across four systems: original MAPF-GPT 2M, original policy with CS-PIBT shield, fine-tuned policy π^* , and fine-tuned policy with shield. The shield improves reliability substantially, while fine-tuning provides standalone gains and further improves the shielded deployment setting in dense warehouse scenarios.

C. Uncertainty-Aware Planner

We evaluate three configurations on the POGEMA benchmark using the 01-random and 02-mazes evaluation suites with agent counts $N \in \{8, 16, 24, 32, 48, 64\}$: (1) **Vanilla**: the pre-trained MAPF-GPT-2M model with no wrapper; (2) **Vanilla + Safe**: the same model with the uncertainty-aware planner; (3) **Finetuned + Safe**: the fine-tuned policy π^* with

TABLE III
PERFORMANCE COMPARISON AVERAGED OVER RANDOM AND MAZE SUITES. CSR AND ISR ARE REPORTED AS FRACTIONS; COLLISIONS ARE MEAN PER EPISODE.

N	Method	CSR	ISR	Coll.	Time (s)
32	Vanilla	0.76	0.96	56.4	2.1
	Van.+Safe	0.95	0.99	34.2	5.9
	FT+Safe	0.93	0.99	18.8	6.4
48	Vanilla	0.52	0.82	233.8	4.4
	Van.+Safe	0.84	0.97	138.4	12.7
	FT+Safe	0.85	0.96	81.3	13.3
64	Vanilla	0.32	0.71	573.7	6.8
	Van.+Safe	0.56	0.85	455.2	22.4
	FT+Safe	0.68	0.90	249.5	22.3

the uncertainty-aware planner. Both planner configurations use $\lambda_3 = 0.3$, horizon $s = 3$, pruning threshold $\epsilon = 0.1$, decay $\alpha = 0.5$, $\lambda_1 = \lambda_2 = 1.0$, and visit window $W = 10$. Results are averaged across both suites.

Table III and Figure 3 show that the vanilla MAPF-GPT-2M policy degrades sharply with agent density.

1) *Planner with vanilla policy*: Applying the uncertainty-aware planner to the vanilla model yields large improvements, particularly in the mid-density regime. At 24 agents, CSR rises from 89% to 100%; at 32 agents, from 76% to 95%. At 64 agents, CSR increases from 32% to 56% and collisions decrease from 574 to 455 (21% reduction). The planner is most effective on random maps, where it achieves 86% CSR at 64 agents (vs. 58% vanilla), but also helps on mazes (26% vs. 6%). Runtime overhead is approximately 3 $\times$ the vanilla baseline.

2) *Planner with fine-tuned policy*: Combining the fine-tuned policy π^* with the planner further improves results. CSR improves from 56% (vanilla+safe) to **68%** at 64 agents and from 84% to **85%** at 48 agents. Up to 24 agents, the system achieves **100% CSR** on both map types. Mean collisions per episode at 64 agents drop from 455 to **250** (45% reduction versus vanilla+safe, 56% versus vanilla alone).

The improvement is synergistic: the fine-tuned policy provides both better individual actions *and* a more accurate model for simulating higher-priority neighbors during risk-map construction. The gains are most pronounced on maze maps at high density: at 64 agents, CSR rises from 26% (vanilla+safe) to 45% (finetuned+safe). Runtime is comparable to vanilla+safe (22.3s vs. 22.4s at 64 agents), since the improved policy resolves conflicts earlier, offsetting the overhead of more accurate simulation.

We conduct an ablation study on the three cost-function weights λ_1 , λ_2 , and λ_3 (Eq. 5), sweeping each while holding the others at their default values.

For CSR, λ_1 performs best in the range 1.0–2.0, $\lambda_2 = 2$ appears to be a sweet spot, and $\lambda_3 = 0.3$ yields the highest success rate. For collision count, the trends are clearest at 64 agents, where conflicts are frequent: collisions decrease monotonically with increasing λ_2 and λ_3 , while λ_1 saturates around 2.0. At 32 agents, collision counts are already low across all settings, making the differences less pronounced.

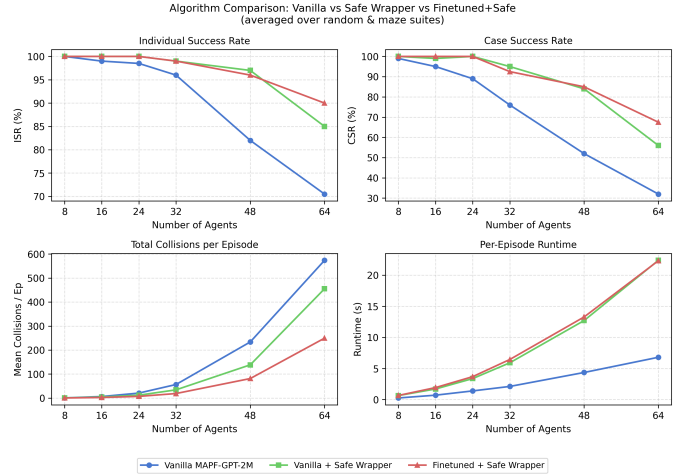


Fig. 3. ISR, CSR, collisions, and runtime across agent counts for the three configurations, averaged over random and maze suites.

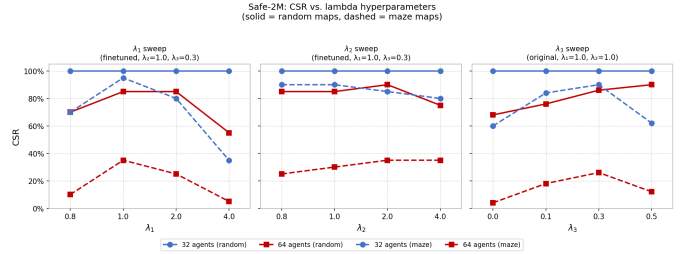


Fig. 4. CSR across different λ

These results confirm that the risk map (weighted by λ_2) contributes to both higher success rates and fewer collisions, and that the visited-state penalty (λ_3) is important for breaking livelocks in dense scenarios. The policy-preference weight λ_1 provides diminishing returns beyond 2.0, suggesting that over-relying on the learned policy’s action distribution can suppress the planner’s corrective signals.

VI. CONCLUSION

We presented a communication-free decentralized MAPF system targeting environments where inter-agent communication is denied. Our approach combines two complementary techniques. First, we collect hard states using a centralized CS-PIBT shield and fine-tune MAPF-GPT-2M on the shield’s corrective actions, converting reactive shielding into proactive policy improvement. Second, we introduce an uncertainty-aware planner that exploits a shared policy assumption to simulate higher-priority neighbors, construct a cumulative risk map, and apply a visited-state penalty to avoid livelock.

Our results (Section V) show that fine-tuning alone substantially reduces collisions and improves completeness, while the planner alone provides strong gains in congested settings—raising CSR well above vanilla MAPF-GPT. The combination of both achieves the best overall performance, reaching 68% CSR at 64 agents compared to 32% for the vanilla baseline.

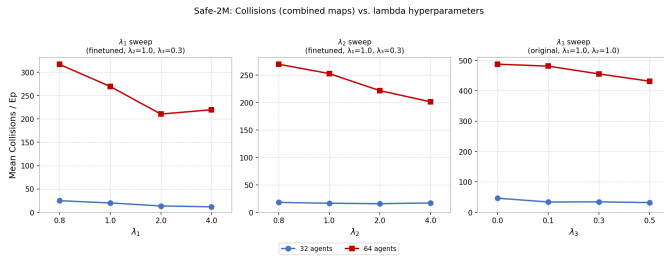


Fig. 5. Collision across different λ .

Several limitations remain. First, the system provides no formal safety or completeness guarantees: because simulation is limited to the local observation radius, deadlocks and collisions can still occur in highly congested scenarios. Second, the runtime of the uncertainty-aware planner grows with agent density, reaching approximately $3\times$ the baseline at 64 agents. Third, we assume perfect perception within the local field of view.

A. Future Work

In future work, we plan to evaluate the system under partial observability and noisy sensing, explore online priority adaptation to reduce deadlocks, and investigate whether larger fine-tuned models can further close the gap on maze maps.

B. Contributions of Each Team Member

1) *Thomas*: Thomas implemented the CS-PIBT shield for data collection, and designed and developed the uncertainty-aware planner.

2) *Chiawen*: Chiawen collected data using the CS-PIBT shield and performed data filtering to identify informative hard states for fine-tuning. Chiawen also developed the MAPF-GPT fine-tuning pipeline.

REFERENCES

- [1] Anton Andreychuk, Konstantin Yakovlev, Aleksandr Panov, and Alexey Skrynnik. Mapf-gpt: Imitation learning for multi-agent pathfinding at scale, 2025. URL <https://arxiv.org/abs/2409.00134>.
- [2] Anton Andreychuk, Konstantin Yakovlev, Aleksandr Panov, and Alexey Skrynnik. Advancing learnable multi-agent pathfinding solvers with active fine-tuning, 2025. URL <https://arxiv.org/abs/2506.23793>.
- [3] Somil Bansal, Mo Chen, Jaime F Fisac, and Claire J Tomlin. Safe sequential path planning under disturbances and imperfect information. *arXiv preprint arXiv:1603.05208*, 2016.
- [4] Mo Chen, Somil Bansal, Ken Tanabe, and Claire J Tomlin. Provably safe and robust drone routing via sequential path planning: A case study in san francisco and the bay area. *arXiv preprint arXiv:1705.04585*, 2017.
- [5] Mehul Damani, Zhiyao Luo, Emerson Wenzel, and Guillaume Sartoretti. Primal₂: Pathfinding via reinforcement and imitation multi-agent learning - lifelong. *IEEE Robotics and Automation Letters*, 6(2):2666–2673, April

2021. ISSN 2377-3774. doi: 10.1109/LRA.2021.3062803. URL <http://dx.doi.org/10.1109/LRA.2021.3062803>.
- [6] Haimin Hu, Zixu Zhang, Kensuke Nakamura, Andrea Bajcsy, and Jaime F Fisac. Deception game: Closing the safety-learning loop in interactive robot autonomy. In *Conference on Robot Learning (CoRL)*, 2023.
- [7] Jiaoyang Li, Wheeler Ruml, and Sven Koenig. Eecbs: A bounded-suboptimal search for multi-agent path finding, 2021. URL <https://arxiv.org/abs/2010.01367>.
- [8] Keisuke Okumura. Lacam: Search-based algorithm for quick multi-agent pathfinding, 2022. URL <https://arxiv.org/abs/2211.13432>.
- [9] Keisuke Okumura, Manao Machida, Xavier Défago, and Yasumasa Tamura. Priority inheritance with backtracking for iterative multi-agent path finding. *Artificial Intelligence*, 310:103752, September 2022. ISSN 0004-3702. doi: 10.1016/j.artint.2022.103752. URL <http://dx.doi.org/10.1016/j.artint.2022.103752>.
- [10] Lasse Peters, Andrea Bajcsy, Chih-Yuan Chiu, David Fridovich-Keil, Forrest Laine, Laura Ferranti, and Javier Alonso-Mora. Contingency games for multi-agent interaction. *IEEE Robotics and Automation Letters*, 2024.
- [11] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R. Sturtevant. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence*, 219:40–66, 2015. ISSN 0004-3702. doi: <https://doi.org/10.1016/j.artint.2014.11.006>. URL <https://www.sciencedirect.com/science/article/pii/S0004370214001386>.
- [12] Rishi Veerapaneni, Arthur Jakobsson, Kevin Ren, Samuel Kim, Jiaoyang Li, and Maxim Likhachev. Work smarter not harder: Simple imitation learning with cs-pibt outperforms large scale imitation learning for mapf, 2024. URL <https://arxiv.org/abs/2409.14491>.
- [13] Rishi Veerapaneni, Qian Wang, Kevin Ren, Arthur Jakobsson, Jiaoyang Li, and Maxim Likhachev. Improving learnt local mapf policies with heuristic search, 2024. URL <https://arxiv.org/abs/2403.20300>.
- [14] Rishi Veerapaneni, Jiaoyang Li, and Maxim Likhachev. Anytime single-step MAPF planning with anytime PIBT, 2025. URL <https://arxiv.org/abs/2504.07841>.
- [15] Rishi Veerapaneni, Muhammad Suhail Saleem, Jiaoyang Li, and Maxim Likhachev. Windowed mapf with completeness guarantees. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(22):23323–23332, April 2025. ISSN 2159-5399. doi: 10.1609/aaai.v39i22.34499. URL <http://dx.doi.org/10.1609/aaai.v39i22.34499>.
- [16] Yutong Wang, Bairan Xiang, Shinan Huang, and Guillaume Sartoretti. Scrimp: Scalable communication for reinforcement- and imitation-learning-based multi-agent pathfinding, 2023. URL <https://arxiv.org/abs/2303.00605>.
- [17] Hongzhan Yu, Chiaki Hirayama, Chenning Yu, Sylvia Herbert, and Sicun Gao. Sequential neural barriers for scalable dynamic obstacle avoidance. 2023.